Dynatrace SNMP at Scale — Engineering Report

Cloud test bed · tenant `qof78400` · Azure `rg-snmp-testbed`

2026-11-12

Executive summary

This report covers an end-to-end investigation into running Dynatrace SNMP monitoring and discovery at retail scale (**9,000+ stores**, target **~90,000 ICMP tests** and **~20,000 SNMP devices**), validated on a purpose-built Azure test bed.

Four findings drive every recommendation:

1. **Discovery scales easily; monitoring is the real load.** SNMP autodiscovery sweeps ~**117 IP/s** on one 8-vCPU ActiveGate. A 9,000-address store sweep takes **~77 seconds on a single AG**. By contrast, *monitoring* 20,000 devices is a fleet problem whose binding constraint is **metric volume and entity count (~0.5 M entities, ~1.1–5.5 B datapoints/day)**, not CPU.

2. **The `.252` fixed-last-octet enumeration is the key batching technique.** It collapses a full `10/8` (16.7 M addresses) to **65,536** — a 256× reduction that makes /8-scale coverage a single-AG job. Fixed last octets are not CIDR-expressible, so configs must enumerate `/32`s within hard limits of 100 addresses/group, 100 groups/config, 100 configs/AG-group.

3. **Simulators can prove throughput but *cannot* prove topology.** `EXT_NETWORK_DEVICE` identity is derived from `chassis.mac`, which is an ActiveGate **datasource property, not an SNMP OID**. Simulated devices therefore all collapse into a single Smartscape entity regardless of how many you run. Topology fidelity requires real devices.

4. **ICMP sizing is a scheduling question, not a packet question.** 90,000 probes/minute is ~1,500 pps — trivial. The limit is synthetic **execution slots**. Schema limits (10,000 targets per request, 100 requests per monitor) mean batching choices change required capacity by ~10×.

Biggest caveat: every throughput number was measured intra-VNet at sub-millisecond RTT. A 9,000-store estate is WAN-bound (20–100 ms RTT, jitter, loss), and SNMP sweeping is latency-bound, not CPU-bound. **Treat lab figures as an optimistic ceiling.**

Most important open item: the customer already runs **~2,000 stores × 2–3 tests in production**. That real-WAN deployment is better sizing evidence than any lab extrapolation, and analysing it should precede further lab work.

1. Test bed as built

Component	Spec	Role
`vm-ag`	D4s_v5, Linux x86-64, AG 1.349, EEC profile HIGH	ActiveGate, group `cloud-snmp`
`vm-snmpsim`	D8s_v5, Ubuntu 22.04, snmpsim-lextudio 1.1.1	Simulator; owns routed `10.20.64.0/18`
`vm-c8000v`	D4s_v5, Cisco Catalyst 8000V 17.15.6 (BYOL)	Real SNMP device (fidelity)
`vm-fgt`	D4s_v5, FortiGate-VM	Real device (SNMP never configured)

Network: VNet `10.20.0.0/16`, subnet `snet-infra` `10.20.0.0/24`, UDR `rt-sim` routing `10.20.64.0/18` to the simulator (next-hop `10.20.0.5`), NIC IP-forwarding enabled.

Extensions: `snmp-auto-discovery` **4.3.4**, `snmp-generic-device` **3.1.5**.

1.1 Environment corrections worth knowing

- **Extension 4.3.4 is required on current AGs.** AG 1.349 ships a **Python 3.14** datasource runtime; 2.4.7 pins `<3.11` and fails to build its venv; 3.0.0 builds but crashes on any responding device (`utils.py:177 getOid()` — `AttributeError: 'ObjectIdentifier' object has no attribute 'getOid'`). Documented separately as a bug report.
- **4.x requires a platform token** (`dt0s…`) with `storage:buckets:read` + `storage:smartscape:read` in `scanner.token`. A classic `dt0c01` token returns HTTP 401 and the scan never promotes past fastcheck.
- **Scanner uses `profile`** (`DEFAULT` / `HIGH` / `DEDICATED`), not `threads` (that was 2.4.x).
- **`frequency` must exceed `scan_duration`**, or every cycle logs `run_in_thread … longer than the interval`.

2. Measured throughput — SNMP autodiscovery

10,000 `/32` targets per config; metric `sfm.snmp_autodiscovery.scan_duration`.

AG size	`scanner.profile`	Workload	devices_ok	scan_duration	per-IP	high-CPU
D2s_v5 (2 vCPU)	DEFAULT	all-respond	10,000	~200 s	0.020 s	no
D2s_v5 (2 vCPU)	DEFAULT	timeout	0	303 s	0.030 s	**fired**

AG size	`scanner.profile`	Workload	devices_ok	scan_duration	per-IP	high-CPU
D8s_v5 (8 vCPU)	DEFAULT	timeout	0	286 s	0.029 s	none
D8s_v5 (8 vCPU)	**DEDICATED**	timeout	0	**85.6 s**	**0.0086 s**	fired once

Interpretation

- **Empty-space sweeps are I/O-bound, not CPU-bound.** 2→8 vCPU barely changed duration (303→286 s) but cleared CPU saturation. **Cores buy headroom; concurrency buys speed.**
- `DEDICATED` **is the speed lever** — 3.3× faster, ~**117 IP/s**. Safe only once ≥8 vCPU provides headroom; on 2 vCPU it worsens the CPU alert.
- **Responding devices are cheaper per IP than timeouts**, so a live store estate is the easy case; size on the timeout rate as the pessimistic bound.

/8 sizing at the best setting

Target	Addresses	1-AG sweep	AGs @ 12 h	AGs @ 24 h
`.252` enumeration (`10.*.*.252`)	65,536	**~9 min**	**1**	1
Full `/8` (every address)	16,777,216	~40 h	~4 *	~2 *

* Throughput only. The **100 configs × 10,000 = 1,000,000 addresses per AG-group** limit means a full /8 needs ≥17 AG-groups' worth of configs regardless of speed. **The `.252` enumeration is the recommended path.**

3. Autodiscovery configuration for fixed last octets

3.1 Hard limits

Limit	Value
Addresses per group	**100**
Groups per config	**100**
→ Addresses per config	**10,000**
Configs per AG group	**100**
→ Addresses per AG group	**1,000,000**

A fixed last octet (`.252`, `.250`, …) is **not expressible as a CIDR**; every target must be an enumerated `/32`. `10.[0-255].[0-255].252` = 256 × 256 = **65,536** addresses.

3.2 Preferred: enumerate known store subnets

9,000 stores = 9,000 `/32` entries → **fits in ONE config** (90 groups × 100) and sweeps in ~77 s. Blind-sweeping 65,536 to find 9,000 devices is 7× the work for no additional information when the store list exists in IPAM/CMDB/DHCP. **Reserve blind sweeps for discovering unknown stores**, on a slow cadence.

3.3 Blind-sweep layout (aligned and debuggable)

Dense packing yields 7 configs but unaligned ranges. Prefer 32 second-octets per config:

```
config k (k = 0..7):   covers b ∈ [32k .. 32k+31]
  groups:              one per (b, c-chunk); chunks = 0-99, 100-199, 200-255  → 3 per b
  per config:          32 b × 3 = 96 groups (≤100 ✓);  32 × 256 = 8,192 addresses
(≤10,000 ✓)
  total:               8 configs × 8,192 = 65,536 ✓
```

Group labels like `b017-c100` (= `10.17.100-199.252`) make a failing group name its own range.


```
        {"scope":"ag_group-<group>","value":{"enabled":true,"version":"4.3.4",
         "featureSets":["self-monitoring"],"activationContext":"REMOTE",
         "pythonRemote":{"groups":[
           {"label":"b000-c000","addresses":["10.0.0.252/32","10.0.1.252/32","…100 total…"],

            "port":161,"authentication":{"version":"SNMPv2c","useCredentialVault":false,
                                         "community":"<community>"}}],
          "scanner":{"frequency":720,"profile":"DEDICATED","token":"<dt0s… platform token>"}}}}
```


3.4 Multiple last octets

Each additional octet adds 65,536 addresses / 8 configs. Against the 100-configs-per-AG-group ceiling: **up to 12 octets per AG group** (96 configs) with the aligned scheme, 14 with dense packing. Beyond that, add AG groups.

4. Topology constraints

4.1 What creates topology

Component	Creates Smartscape entities?
`snmp-auto-discovery`	**No.** Inventories and classifies; emits metrics only. Measured `devices_ok = 10,000`, 20,001 metric lines, `EXT_NETWORK_DEVICE = 0`.
`snmp-generic-device`	**Yes.** Proven: real C8000V → 1 `EXT_NETWORK_DEVICE` + 3 `EXT_NETWORK_INTERFACE`.

4.2 Entity identity — the binding constraint

Read directly from the 3.1.5 extension bundle:

```
// openpipeline/metrics.pipeline.json
"smartscapeNode": { "nodeType": "EXT_NETWORK_DEVICE",
  "idComponents": [ { "idComponent": "identifier",
                      "referencedFieldName": "chassis.mac" } ] }
```

```
# extension.yaml (~line 359)
- key: chassis.mac
  value: this:device.chassis_mac      # NOT an OID — an AG datasource property
# TODO: Enable with 1.333 release
```

- **`EXT_NETWORK_DEVICE` ID = `chassis.mac`, and nothing else.**
- **`chassis.mac` is not SNMP-readable** — no recording or OID can set it.
- Node **name** comes from `sys.name`; upsert fires on the `sysuptime` metric and requires `isNotNull(sys.name)`.
- **`EXT_NETWORK_INTERFACE` ID = `network_interface_{mac.address}_{if.name}`**, where `mac.address` is explicitly filtered to exclude `n/a` — commented *"required by network topology"*.

Consequence: any device without a resolvable chassis MAC collapses into **one shared entity for any N**, and produces no interface entities.

Evidence

50 simulated devices, each given a unique `sysName`, `sysDescr`, `sysObjectID`, `sysContact`, `sysLocation`, `sysUpTime`, `ifPhysAddress`, `dot1dBaseBridgeAddress` and `lldpLocChassisId`, polled cleanly (`Endpoints OK: 50 WARNING: 0 ERROR: 0`) yet produced **exactly one** entity whose attributes rotated per poll:

Observed	name	snmp.ip	sysObjectID
21:40	`sim-dev-024`	–	–
02:47	`sim-device`	10.20.64.2	…8072.3.2.10
03:01	`sim-dev-048`	10.20.64.48	–
03:23	`sim-dev-005`	10.20.64.5	…8072.3.2.5
(global-OUI MACs)	`sim-dev-046`	10.20.64.46	–

`chassis_mac` remained `n/a` throughout, while the real Cisco held a stable separate entity with `chassis_mac: 00:0D:3A:97:49:68`.

4.3 Gates that must pass before any entity appears

1. **Fastcheck must succeed** — `DEVICE_CONNECTION_ERROR` never promotes to polling.
2. **Response source IP must equal the queried IP.** The AG's native `ds:snmp` validates this and silently discards mismatches. *net-snmp and pysnmp do not validate source, which masks the fault.*
3. **The device must answer every OID the enabled featureSets poll.** Sparse tables let a GETBULK overrun into the next column → `INVALID_CONFIG_ERROR: Text-type oid used as metric value`.
4. **`sys.name` must be non-null** (filtered to exclude `n/a`).

4.4 Scale implications

- Topology cost scales with **interfaces, not devices**: 20,000 devices × ~24 ports ≈ **480,000 interface entities + 20,000 device entities ≈ 0.5 M**.
- Tenant entity ceilings and DDU/licensing bind before CPU.
- Materialization is slow and batchy — minutes per handful, never a clean burst.
- Interface count is reducible via config vars `if_name`, `if_oper_status`, `if_admin_status`.

5. SNMP monitoring load (`snmp-generic-device`)

Continuous polling (ifTable/ifXTable walks per interval), not a one-shot probe.

Only lab datapoint: 50 devices × 2 interfaces (100 interfaces), `SNMP Health` + `Interfaces`, 1-minute interval → **loadavg 0.14 on 4 vCPU** (~0.14 cores per 100 interfaces).

> **Do not fleet-size from this.** At loadavg 0.14 most cost is fixed EEC/datasource baseline, not marginal per-interface work. Naive linear scaling (implying 100+ AGs for 480,000 interfaces) is unreliable in both directions. **A pilot at 500–1,000 real devices on one AG is required.**

Metric volume is calculable and likely binding:

Assumption	1-min interval	5-min interval
20,000 devices × 24 interfaces = 480,000 interfaces, ~8 series each	~3.8 M dp/min ≈ **5.5 B/day**	~0.77 M dp/min ≈ **1.1 B/day**

Three levers, in order of impact:

1. **Interval** — 1→5 min cuts volume 5×.
2. **Interface filtering** — `if_name`, `if_oper_status`, `if_admin_status` config vars skip down/unused ports, often the majority on retail switches.
3. **featureSets** — enable only what is alerted on; never enable `Interfaces 32-bit` and `64-bit` together.

6. ICMP at scale

6.1 Mechanism

ICMP availability = **“multiprotocol” synthetic monitors**, configured via Settings 2.0 schema `builtin:synthetic.multiprotocol.config` (siblings `.name`, `.scheduling`, `.outage-handling`, `.performance-thresholds`). The config is scoped to `MULTIPROTOCOL_MONITOR`, so **monitors must be created via the synthetic API first**, then configured as a settings object.

6.2 Batching envelope (from the schema)

Limit	Value
`Step.targetList` — targets per request	**10,000**
`properties.steps` — requests per monitor	**100**
→ theoretical max per monitor	**1,000,000 targets**
`frequency`	per-monitor; one of **1, 2, 5, 10, 15, 30, 60** minutes

6.3 Scheduling model — important distinction

Synthetic monitors are independently scheduled, each with its own frequency. They are *not* a serial queue: a target’s retest interval equals its monitor’s frequency, regardless of how many other monitors exist — **provided the AG has capacity**.

Capacity condition for *M* monitors at frequency *F* minutes, execution time *T* seconds, *C* concurrent slots:

```
C / T  >=  M / (60 x F)
```

- *F*=5, *T*=2 s → ~3.3 slots needed (trivial)
- *F*=5, *T*=100 s → ~167 slots needed (saturation)

When saturated, executions go **overdue** and the effective interval stretches to `total work ÷ capacity`. That is the only circumstance producing the feared “10-minute lag”.

> **Do not carry over the SNMP rule.** For autodiscovery, `frequency` *must* exceed `scan_duration` because one config genuinely is a serial sweep. Synthetics do not work that way.

6.4 Working ICMP step template

Lifted verbatim from the live monitor `MULTIPROTOCOL_MONITOR-486A47D176F7FCF2`:

```
        {"requestType":"ICMP","targetList":["10.20.64.1","…"],
         "properties":[{"key":"EXECUTION_TIMEOUT","value":"PT1S"},
                       {"key":"ICMP_TIME_TO_LIVE","value":"64"},
                       {"key":"ICMP_PACKET_SIZE","value":"32"},
                       {"key":"ICMP_NUMBER_OF_PACKETS","value":"1"},
                       {"key":"ICMP_TIMEOUT_FOR_REPLY","value":"PT1S"},
                       {"key":"ICMP_DO_NOT_FRAGMENT_DATA","value":"true"},
                       {"key":"ICMP_IP_VERSION","value":"4"}],
         "constraints":[{"type":"SUCCESS_RATE_PERCENT",
           "properties":[{"key":"operator","value":">="},{"key":"value","value":"80"}]}],

         "requestConfigurations":[{"constraints":[{"type":"ICMP_SUCCESS_RATE_PERCENT",
           "properties":[{"key":"operator","value":">="},{"key":"value","value":"80"}]}],

           "properties":[]}]}
```

That template had **one** target with `EXECUTION_TIMEOUT: PT1S`. With 100 targets per step the execution timeout will likely need raising (start at `PT10S` and tune down from observed duration).

6.5 Endpoint simulation for ICMP

The simulator's kernel `local` route makes it answer ICMP for **every** address in its routed range — **16,384 free responders** in `10.20.64.0/18`, verified from the AG (`10.20.64.1`, `10.20.100.7`, `10.20.127.254` all reply). No per-IP configuration and no snmpsim involvement. Reaching ~50,000 requires routing `10.20.128.0/17` and `10.20.32.0/19` to the simulator (UDR + a matching `ip route replace local … dev lo`).

A generator (`provisioning/gen-icmp-synthetics.py`) emits 500 monitors × 100 targets = 50,000, plus a flat target list for `fping` verification.

6.6 Production supersedes the lab

The customer already runs **~2,000 stores × 2–3 tests** in production on real WAN. Scaling to the target is **not** a 4.5× problem:

	Stores	Tests/store	Total ICMP tests
Today	~2,000	2–3	~5,000
Target	9,000	~10	90,000
Multiplier	4.5×	~4×	**~18×**

The per-store test depth is the hidden multiplier. Five numbers from production convert this to arithmetic: AG/location count and headroom; configured frequency; execution duration p50/p95; whether any executions are already overdue; and whether the 2–3 tests are separate *requests* or multiple targets in one `targetList`.

Batching lever: 10 tests/store as 10 separate requests → 90,000 executions. As **one request with a 10-entry** `targetList` → 9,000 executions for identical coverage — ~10× less scheduling overhead.

7. Heuristics

Autodiscovery sweeping

1. **~117 IP/s per 8-vCPU AG at** `DEDICATED` (timeout-bound, lab LAN). Sizing: `seconds ≈ addresses ÷ 117`. Halve over WAN until measured.
2. **Concurrency buys speed; cores buy headroom.**
3. `DEDICATED` **only at ≥8 vCPU.**
4. `frequency` **must exceed** `scan_duration` — target ≥2× margin.
5. **Enumerate known subnets over blind sweeps.**
6. **Stagger config start times** so many configs don't fire simultaneously.
7. **Size on the timeout rate**, which is the pessimistic case.
8. `.252` **enumeration collapses a /8 by 256×** — the single most valuable technique here.

SNMP monitoring

1. **Size by interfaces, not devices.**
2. **Levers in order:** interval → interface filters → featureSets.
3. **Devices must answer every OID an enabled featureSet polls.**
4. **Entity identity needs a real chassis MAC** — fine for real gear, fatal for simulators.

ICMP / synthetics

1. **Monitors are independently scheduled**; total sweep time is not a scheduling floor.
2. **Batch targets into** `targetList` rather than multiplying monitors.
3. **Raise** `EXECUTION_TIMEOUT` when a step carries many targets.
4. **Ramp 50 → 150 → 500** and watch for overdue executions — that knee is the sizing number.

Both

1. **Dedicated AG groups per role** — sweeps and monitoring must not share an AG.
2. **Budget entities and DDUs first**; they decide the architecture before CPU does.
3. **Calibrate against one real region** before committing a fleet size.

8. Recommendations

1. **Use the** `.252` **enumeration**, and prefer enumerating known store subnets (one config, ~77 s) over blind sweeps.
2. **Separate the AG roles**: a small sweep AG group (1–2 AGs at 8 vCPU, `DEDICATED`) and a separately-sized monitoring fleet.

3. **Decide monitoring depth before sizing.** 20,000 devices at 1-minute intervals with full interface coverage is ~5.5 B datapoints/day; at 5-minute intervals with interface filtering it is a fraction of that. This choice dominates cost.
4. **Validate per-store ICMP depth.** Going from 2–3 to ~10 tests per store is a 4× multiplier on top of the 4.5× store growth. Batch with `targetList` wherever coverage allows.
5. **Analyse the existing 2,000-store production deployment** before further lab work — it is the only real-WAN evidence available.
6. **Use the lab for the breaking point, not the operating point.** Production cannot safely be driven to saturation; the test bed can, and that knee is what converts production measurements into a defensible AG count.
7. **Do not attempt to validate topology with simulators.** Use real devices for topology fidelity; use the simulator for discovery throughput only.

Appendix A — Open items

#	Item	Why it matters
1	ICMP per-AG ceiling (execution slots)	Largest unknown; 90,000 tests planned
2	Per-AG device ceiling for `generic-device` at 500–1,000 real devices	Fleet sizing input
3	WAN-latency effect on sweep throughput	All lab numbers are sub-ms LAN
4	Tenant entity + DDU ceilings for ~0.5 M entities	Likely the true architectural limit
5	Re-test `chassis.mac` on a newer extension	`TODO: Enable with 1.333 release` suggests change

Appendix B — Repository artifacts

Path	Contents
`docs/cloud-snmp-testbed-plan.md`	Test bed plan and measured throughput campaign
`docs/50k-topology-sizing.md`	Topology identity root cause and 50k feasibility
`docs/9000-store-sizing-and-configs.md`	Load sizing, config layouts, heuristics
`docs/handoff-state.md`	Session-to-session state and environment gotchas
`docs/dynatrace-snmp-auto-discovery-3.0.0-bug.md`	`getOid()` crash bug report
`provisioning/azure-snmp-testbed.sh`	Idempotent test-bed build script

Path	Contents
`provisioning/gen-icmp-synthetics.py`	50k ICMP monitor payload generator

Appendix C — Three blockers encountered (methodological note)

Two were solved and one proved structural. All three initially presented as the *same* symptom — “devices don’t appear” — which is why each needed isolating before the next became visible.

#	Blocker	Resolution
1	AG native `ds:snmp` rejects replies whose source IP ≠ queried IP	**Solved** — bind the simulator to specific destination IPs
2	Recording must cover every featureSet OID (missing `ifXTable`)	**Solved** — full ifTable + ifXTable
3	Entity ID = `chassis.mac`, a datasource property not an OID	**Structural** — simulation cannot satisfy it

A misdiagnosis is worth recording: blocker 1 was initially attributed to a “pysnmp GETBULK incompatibility” because manual `snmpget`/`snmpbulkget` succeeded where the extension failed. Both net-snmp and pysnmp accept replies from a mismatched source address; only the AG’s native datasource validates it. **Reproducing with a different client library than the one under test can produce a confidently wrong conclusion** — a packet capture settled it in minutes.